

Practical 4

Dr Simon Denman
CAB420: Machine Learning

This weeks practical will focus on deep neural networks. You will create your own networks, inspired by the structure of existing networks from the literature, and apply them to a classification task. **Please note that neither of the architectures that you will implement in this practical are state-of-the-art. Both have been supplanted by many other methods, but they are good starting points.**

You will train and evaluate your model on the *CIFAR-10* database. *CIFAR-10* database is an MNIST like database consisting of small (32×32 pixel) colour, natural world images. Like MNIST, it contains 10 classes, and is a balanced dataset.

Problem 1. Implementing an AlexNet-Like Network. Deep Neural Networks have exploded in popularity in recent years due to their impressive performance. For the most part, neural networks are very adaptable and a network that performs well on one task will perform similarly well on another similar task (here, we can consider both tasks being image classification to make them sufficiently similar), assuming that other factors such as dataset characteristics are similar. Given this, it is very common to see the same, or very similar architecture used over and over again. These architectures are typically based on large models that have performed well (or won) a popular image classification benchmark, the ImageNet Large Scale Visual Recognition Challenge (ILSVRC).

The first deep network to win the ILSVRC challenge was AlexNet [1]. The structure of AlexNet is shown in Figure 1.

This network consists of:

- An input, of size $224 \times 224 \times 3$ (i.e. a 224 pixel wide, by 224 pixel high, colour image).
- A 2D Convolution layer with 96 11×11 sized filters, followed by max pooling layer that operates over a 4×4 region.
- A 2D Convolution layer with 256 5×5 sized filters, followed by a max pooling layer.
- A 2D Convolution layer, with 384 3×3 size filters.
- A 2D Convolution layer, with 384 3×3 size filters.
- A 2D Convolution layer, with 256 3×3 size filters.
- A fully connected (or dense) layer with 4096 units.
- A fully connected (or dense) layer with 4096 units.

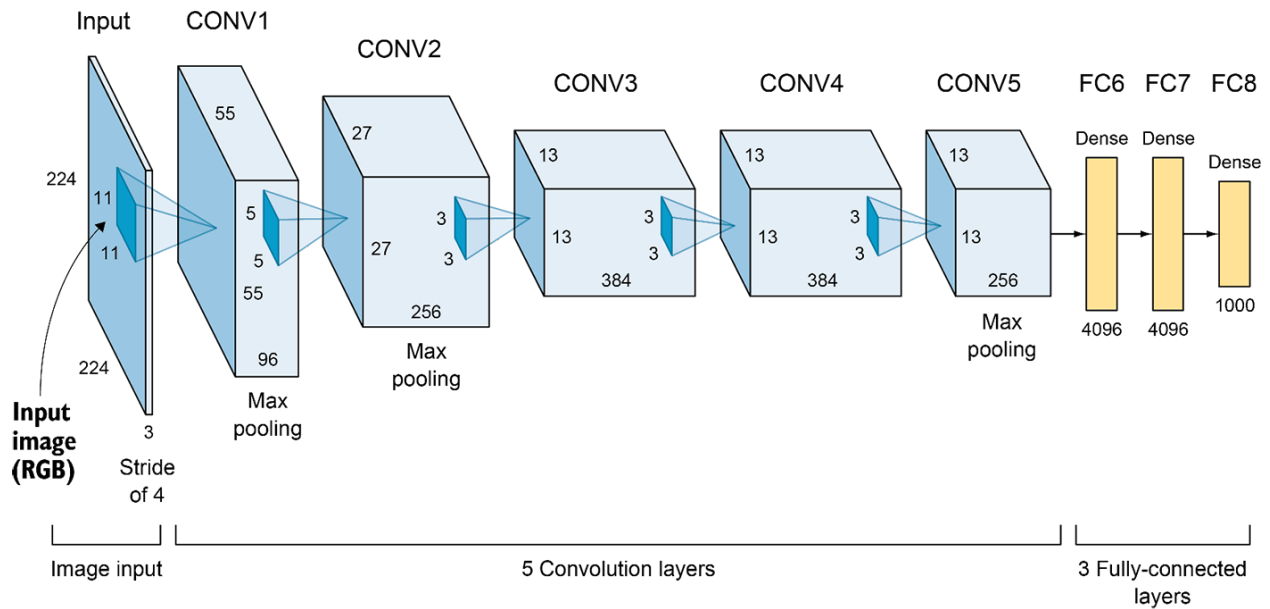


Figure 1: The AlexNet architecture, taken from <https://livebook.manning.com/book/grokking-deep-learning-for-computer-vision/chapter-5/v-3/>. AlexNet has five convolution layers, and 3 fully connected layers.

- An output fully connected (or dense) layer, with 1000 units (for the 1000 classes in ImageNet).

Using this network on the CIFAR-10 dataset is not practical, but we can adapt the AlexNet architecture to CIFAR-10. You are to create a network of a similar style AlexNet [1], but tailored for CIFAR-10. A suggested approach would be:

- Use three 2D convolution layers (with max pooling in between), with filters of decreasing size, but increasing in number in similar ratios to AlexNet (i.e. the first convolution layer has 8 7×7 filters, the second has 24 5×5 filters, the third has 32 3×3 filters). Use 4×4 max pooling after the first layer, and 2×2 after the second.
- Use two dense (fully connected) layers, one with 512 units after the convolution layers, and then a final classification layer with 10 units (i.e. the number of output classes).
- Use relu activations throughout, except for your output layer which should use a softmax layer.

Train the network on the CIFAR-10 dataset, and evaluate the performance. When training the model, be sure to consider the dataset size (50,000 training samples) and batch size when selecting how many epochs to train for. Assuming you're using an optimiser such as Adam with default values, ≈ 5000 iterations will likely be sufficient to train your network to near convergence. After training, plot the training and validation losses and accuracies to observe if the model has converged, or is overfitting.

Problem 2. Implementing a VGG-Like Network.

While AlexNet [1] was the first deep convolutional neural network to get widespread attention, it's performance has since been superseded by many other methods. One such method is VGG [2], shown in Figure 2.

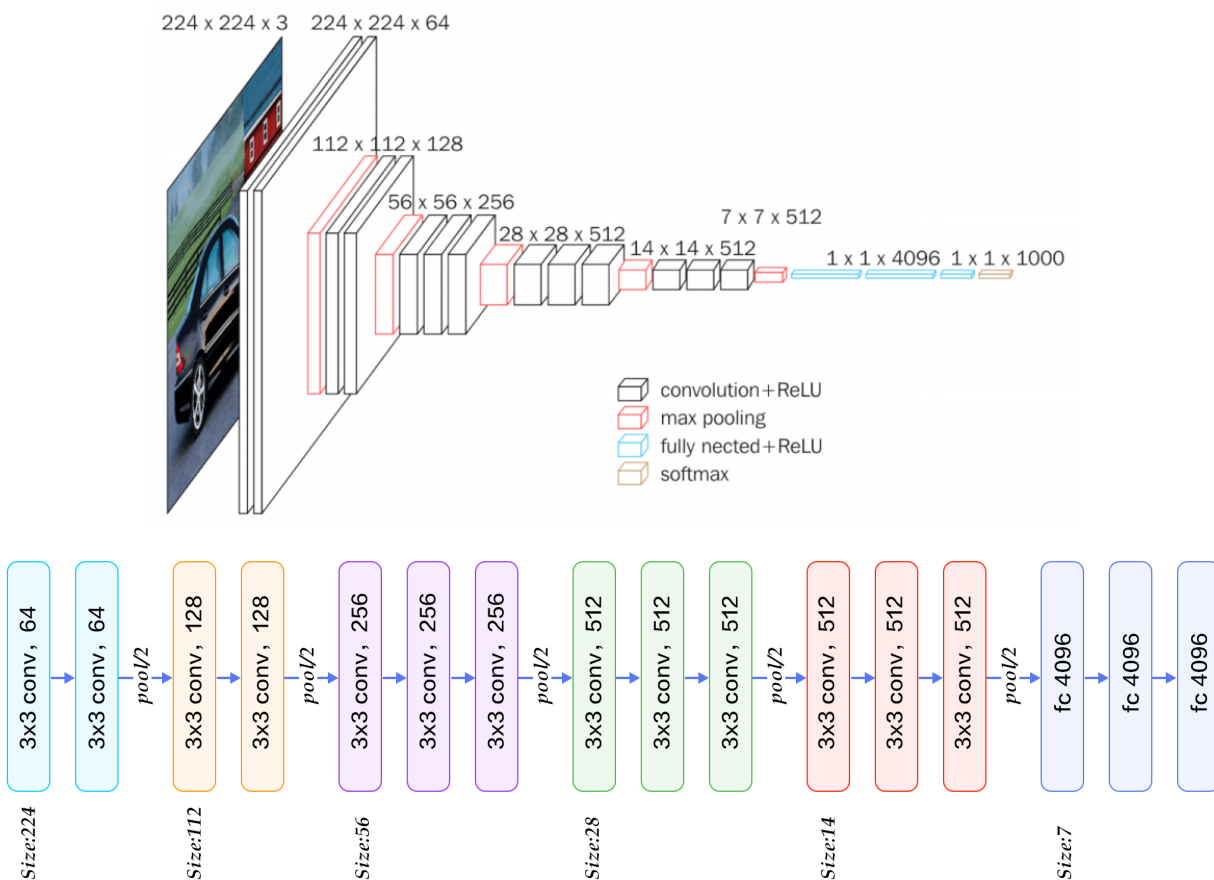


Figure 2: VGG-16 Network Architecture, taken from <https://www.kaggle.com/code/blurredmachine/vggnet-16-architecture-a-complete-guide>. The two images show different ways of explaining the network architecture. The top image makes the representation size at each stage of the network clear, which the bottom representation makes the key parameters of each layer clearer.

VGG-16 (and it's slightly larger brother, VGG-19) makes use of smaller, stacked convolution operations. Where AlexNet used large convolution kernels early in the network (the first layer used a 11×11 kernel, VGG-16 uses only 3×3 kernels, yet stacks these in pairs, or sets of three (or sets of four in VGG-19).

Using the above diagrams as a guide, implement a deep convolution neural network to operate on CIFAR-10 that is inspired by VGG-16. A suggested approach would be:

- A pair of 2D Convolution layers (with an activation between), each with $8 \ 3 \times 3$ filters, followed by a max pooling layer.

- A pair of 2D Convolution layers (with an activation between), each with 16 3×3 filters, followed by a max pooling layer.
- A pair of 2D Convolution layers (with an activation between), each with 32 3×3 filters, followed by a max pooling layer.
- Two dense (fully connected) layers, one with 512 units after the convolution layers, and then a final classification layer with 10 units.
- Use relu activations throughout, except for your output layer which should use a softmax layer.

Train and evaluate the network on CIFAR-10, and compare the performance to the earlier trained AlexNet style network. When comparing the networks, also consider the number of parameters in each network, how long the two networks took to train and evaluate, and what was observed with respect to model convergence and overfitting.

Problem 3. Exploring Other Layers. There are many changes or extensions that you could make to the above networks to further explore deep convolutional neural networks. In your own time, try the following:

- Add dropout to one (or both) networks in a few locations. Use `SpatialDropout2D` layers after convolution layers, and `Dropout` layers after dense layers. With dropout layers in the network, consider:
 - How is the number of iterations required to train the network to convergence impacted by the addition of dropout?
 - Are any differences with respect to overfitting observed?
 - What happens as the dropout probability is increased?
- Add Batch Normalisation to the network in a few locations. With batch normalisation in the network, consider:
 - How is the number of iterations required to train the network to convergence impacted by the addition of batch normalisation?
 - Are any differences with respect to overfitting observed?
- Change the `padding` type used by the convolutional layers. Observe what happens to the representation size when you change this.
- Change the activation used. Consider using an activation such as Swish, or an ELU, rather than the standard relu (though leave the final activation as a softmax). Train the network with the alternate activation and note any changes.

References

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [2] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.